



LECTURE 1

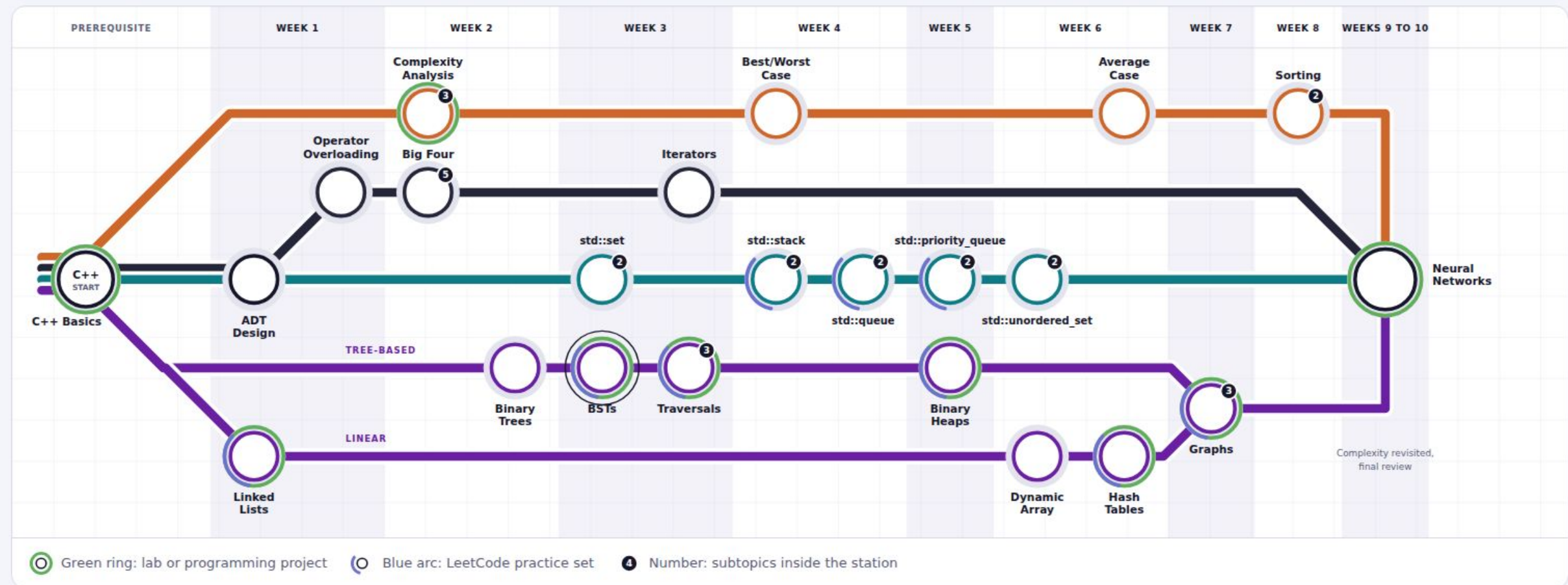
C++ Review + ADT Design

CS24 · Fall 2026 · Diba

CS24 Course Map

Click a station for its subtopics, reading, and coursework.

C++ ADTs Theory Data Structures



WHY THIS COURSE

Why code data structures when AI can do it?

- Coding is an expression of clear understanding and the ability to think from first principles
- Real technical interviews require you to demonstrate your problem-solving skills and articulate your approach
- Caliber: a new practice platform, designed and built by UCSB students, for exactly this

INTRO (2 min) → WHITEBOARD (10 min) → CODING (13 min) → FEEDBACK (5 min)

70% OF YOUR GRADE: PAPER EXAMS + LIVE INTERVIEWS, NO AI

BEFORE WE START

Logistics, fast

- Grading: Practice (LeetCode + Caliber) 10% · Labs + PAs 20% · Quizzes 20% · Mock Interview 15% · Final 35%
- 70% of your grade is paper exams and live coding interviews — no AI there
- Textbook: Savitch. Pre-reading is listed on the course site before each lecture
- Mock interviews run in person. Schedule via Caliber.
- Syllabus: <https://ucsb-cs24.github.io/f26/>

HOW THIS CLASS WORKS

About lectures

- Lectures are problem-solving sessions. Use them to ask questions, discuss with your neighbors, answer live via iClicker
- Why interactive? You learn by doing, like the code tracing activities we will do in a few minutes
- Take a moment: introduce yourself to the people sitting near you
 - your background
 - your CS experience so far, and
 - what you want out of this class

Quick gauge before we start

How comfortable are you with pointers and dynamic memory in C++ right now?

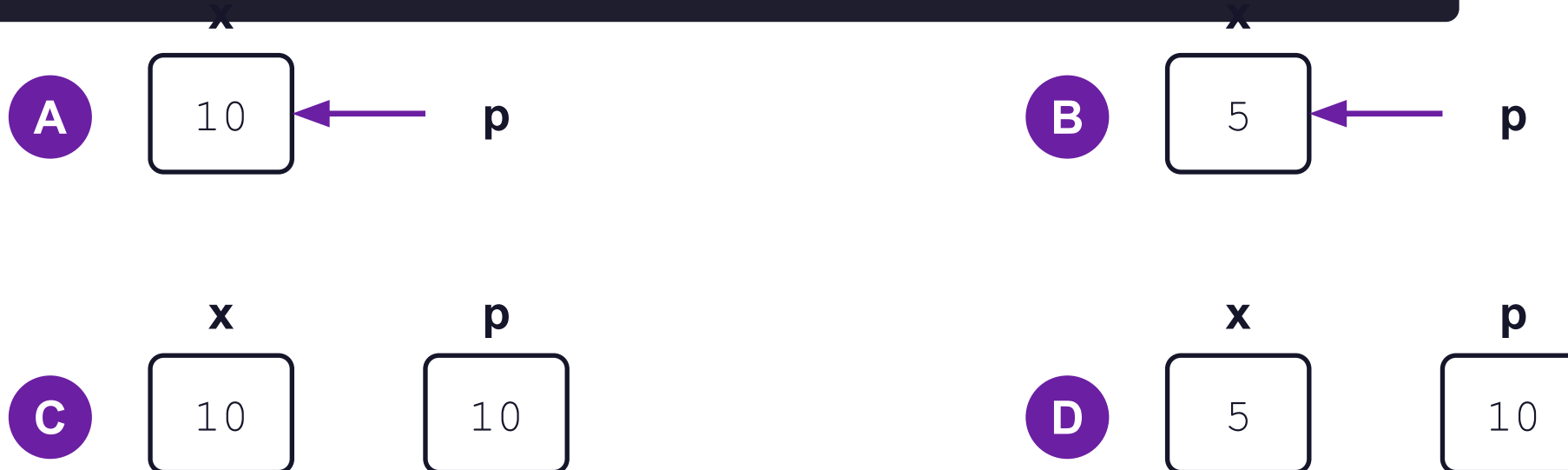
- A. Never used them
- B. Seen them, haven't written much
- C. Comfortable with the basics
- D. Very comfortable, bring on the edge cases



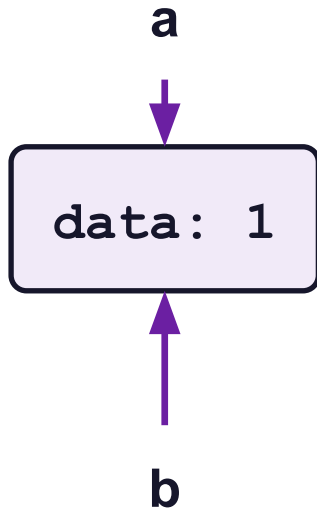
<https://join.iclicker.com/BCRL>

Question 1

```
int x = 5;
int* p = &x;
*p = 10;
// which diagram matches memory now?
```



Question 2

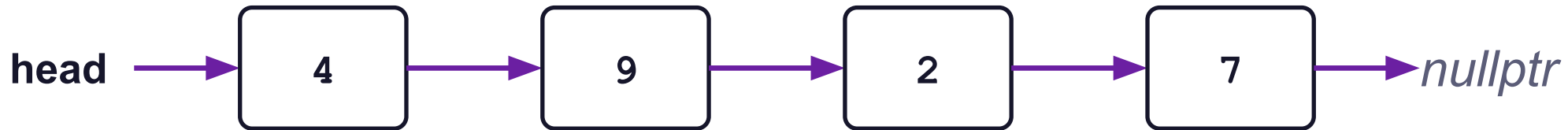


```
Node* a = new Node(1);  
Node* b = a;  
b->data = 99;  
  
// what is a->data now?
```

- A — 1
- B — 99
- C — a crash
- D — undefined

NEW STRUCTURE

Anatomy of a linked list



```
struct Node {  
    int data;  
    Node* next;  
};
```

- head is the only pointer we store
- Everything else: follow ->next until nullptr

Which code adds a node to the head?

```
Node* newNode = new Node(value);
```

A) `newNode->next = head; head = newNode;`

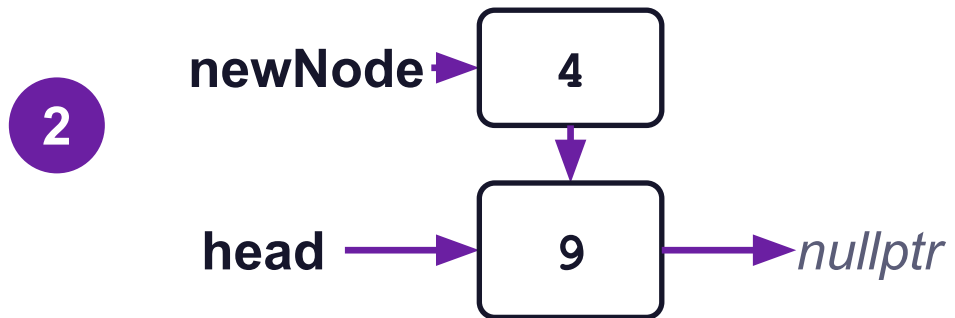
B) `head = newNode; newNode->next = head;`

C) `newNode->next = newNode; head = newNode;`

D) `head->next = newNode;`

WALKTHROUGH

Adding a node to the head



THE PROBLEM

Reverse a linked list

Given the head of a singly linked list,
reverse the list, and return the new head.

Example:

Input: 1 -> 2 -> 3 -> 4 -> 5 -> nullptr

Output: 5 -> 4 -> 3 -> 2 -> 1 -> nullptr

A classic interview question, the same shape as the LeetCode problems you will solve.

<https://leetcode.com/problems/reverse-linked-list/?envType=problem-list-v2&envId=linked-list>

Reverse a Linked List — Recursively

Mapped to the mock-interview rubric

- ☐ Restate + edge case
- ☐ Test case: in → out
- ☐ Recursive or iterative?
- ☐ Derive it — think aloud
- ☐ Name the base case
- ☐ Name the subproblem(s)
- ☐ Combine into the answer
- ☒ Complexity + adapt

(whiteboard your solution)

CONCEPT

What is an ADT (Abstract Data Type)?

- An ADT is how you take a data structure and turn it into something useful. It is defined by what it does, hiding how its implemented.
- Need a collection that grows and shrinks?
Use C++ ADTs: `std::vector` or `std::list`
- Access control (private/public) is how we enforce that — the user of a class should never directly manipulate its data
- **Your goal for Lab01:** turn the chain of nodes into a first-class ADT like `std::list`.

CustomList, our ADT:

- `push_back(value)`
- `print()`

(hides: head, tail, Node internals)

NEW VOCABULARY

Meet Complex

```
class Complex {  
public:  
    Complex(double r, double i);  
    double getReal() const;  
    double getImag() const;  
private:  
    double real, imag;  
};
```

Complex is an ADT for representing numbers of the form $a + jb$.

- a is the real part
- b is the imaginary part

Is the output of this code deterministic?

```
int main() {  
    Complex p;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
private: double real, imag;  
};
```

- A. Yes
- B. No
- C. Not sure

Is the output of this code deterministic?

```
int main() {  
    Complex p;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
private: double real, imag;  
};
```

A. Yes

B. No — the default constructor is empty — p holds junk values

C. Not sure

WHAT IS A CONSTRUCTOR?

Always initialize with a constructor

```
int main() {  
    Complex p;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
    Complex(): real(0), imag(0) {}  
private: double real, imag;  
};
```

WHAT IS A CONSTRUCTOR?

The constructor can take parameters

```
int main() {  
    Complex p(1, 2);  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
    Complex(double re, double im)  
        : real(re), imag(im) {}  
private: double real, imag;  
};
```

Will this code compile?

```
int main() {  
    Complex p;  
    Complex w(1, 2);  
    p = w;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
    Complex(double re, double im)  
        : real(re), imag(im) {}  
private: double real, imag;  
};
```

- A. Yes
- B. No
- C. Not sure

Will this code compile?

```
int main() {  
    Complex p;  
    Complex w(1, 2);  
    p = w;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex {  
public:  
    void conjugate();  
    void print() const;  
    Complex(double re, double im)  
        : real(re), imag(im) {}  
private: double real, imag;  
};
```

B — No — any constructor you write turns off the compiler's free default

WHAT IS A CONSTRUCTOR?

What is the output?

```
int main() {  
    Complex p(1, 2);  
    Complex z(p);  
    z.print();  
}
```

```
class Complex {  
public:  
    void print() const;  
    Complex(double re = 0,  
            double im = 0)  
        : real(re), imag(im) {}  
private: double real, imag;  
};
```

WHAT IS A CONSTRUCTOR?

Every line of code is a function call

```
int main() {  
    Complex p(1, 2);  
    Complex z = p;  
    Complex w(3, 4);  
    z = w;  
    z.print();  
}
```

```
class Complex {  
public:  
    void print() const;  
    Complex(double re = 0,  
            double im = 0)  
        : real(re), imag(im) {}  
private: double real, imag;  
};
```

Critique main.cpp

1 Find at least 3 problems with the flawed CustomList in main.cpp.

- Open main.cpp — this is the version we will NOT keep
- Work with the person next to you, 9 minutes
- Look for: memory issues, slow operations, missing safety checks

ACTIVITY 2 · ON YOUR OWN

Code the constructor, push_back, and print

2

Write all three yourself — use a tail pointer so push_back stays fast.

```
CustomList::CustomList() { /* your code */ }  
void CustomList::push_back(int val) {  
    // your code here  
}  
void CustomList::print() const {  
    // your code here  
}
```

COPY CONSTRUCTOR MEMORY DIAGRAM

```
CustomList list2 = list1;  
  
// ... what just happened?
```

Homework: draw exactly what this looks like in memory.

BEFORE NEXT TIME

Read, answer, draw, and code

- Read: Savitch 4.3–4.4 (operator overloading) and 5.1, 5.3 (constructors/destructors, Rule of Three)
- Write an answer: which function does `cout << x` actually call, and why?
- Write an answer: does `Complex` need a destructor? Why or why not?
- Draw and predict: the diagram for `CustomList list2 = list1` — own nodes, or shared? What problems could arise?
- Code: rewrite `reverse()` for a linked list iteratively (prev/curr, a while loop)